UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 14: Atomic Operations and Histogramming

Volodymyr Kindratenko

3/10/2025

Course Reminders

- Labs
  - Lab 5 due this Friday
  - **Please check your grades for labs 1-4 and make sure they are graded correctly**
- Project
  - Project milestone 1 was due this Friday
  - Project milestone 2 will be released soon
- Midterm 1
  - We are grading it now

Today's Objectives

- To understand atomic operations
  - Read-modify-write in parallel computation
  - A primitive form of "critical regions" in parallel programs
  - Use of atomic operations in CUDA
  - Why atomic operations reduce memory system throughput
  - How to avoid atomic operations in some parallel algorithms
- To learn practical histogram programming techniques
  - Basic histogram algorithm using atomic operations
  - Atomic operation throughput
  - Privatization

A Common Collaboration Pattern

- Multiple bank tellers count the total amount of cash in the safe
  - Each teller grabs a pile and counts
  - Have a central display of the running total
  - Whenever someone finishes counting a pile, add the subtotal of the pile to the running total

- A bad outcome
  - Some of the piles were not accounted for
  - How/why is this possible?

A Common Arbitration Pattern

- Multiple customers booking air tickets
- Each
  - Brings up a flight seat map
  - Decides on a seat
  - Update the seat map, mark the seat as taken

- A bad outcome
  - Multiple passengers ended up booking the same seat
  - How/why is this possible?

Read-Modify-Write Operations

Thread A	Thread B
`oldVal = bins[b]`	`oldVal = bins[b]`
`newVal = oldVal + 1`	`newVal = oldVal + 1`
`bins[b] = newVal`	`bins[b] = newVal`

If bins[b] was **initially 0**, what would the value of bins[b] be after threads 1 and 2 have completed?

- What does each thread get in their Old variable?

Data Races

- A **data race** occurs when multiple threads access the same memory location concurrently without ordering and at least one access is a write
  - Data races may result in unpredictable program output
- To avoid data races, you should use atomic operations.

Data Races Example

Time	Thread A	Thread B
1	`oldVal = bins[b]`	
2	`newVal = oldVal + 1`	
3	`bins[b] = newVal`	
4		`oldVal = bins[b]`
5		`newVal = oldVal + 1`
6		`bins[b] = newVal`

In these two scenarios, the final value of bins[b] will be 2

Time	Thread A	Thread B
1		`oldVal = bins[b]`
2		`newVal = oldVal + 1`
3		`bins[b] = newVal`
4	`oldVal = bins[b]`	
5	`newVal = oldVal + 1`	
6	`bins[b] = newVal`	

Data Races Example

Time	Thread A	Thread B
1	`oldVal = bins[b]`	
2	`newVal = oldVal + 1`	
3		`oldVal = bins[b]`
4	`bins[b] = newVal`	
5		`newVal = oldVal + 1`
6		`bins[b] = newVal`

In these two scenarios (and many others), the final value of bins[b] will be 1

Time	Thread A	Thread B
1		`oldVal = bins[b]`
2		`newVal = oldVal + 1`
3	`oldVal = bins[b]`	
4		`bins[b] = newVal`
5	`newVal = oldVal + 1`	
6	`bins[b] = newVal`	

Mutual Exclusion

- To avoid data races, concurrent read-modify-write operations to the same memory location need to be made **mutually exclusive** to enforce ordering

- One way to do this on CPUs is using **locks** (mutex)
  - Example:

    - `mutex_lock(lock);`
    - `++bins[b];`
    - `mutex_unlock(lock);`

Mutual Exclusion Example

Time	Thread A	Thread B
1	`mutex_lock(lock)`	
2	`oldVal = bins[b]`	
3	`newVal = oldVal + 1`	
4	`bins[b] = newVal`	
5	`mutex_unlock(lock)`	
6		`mutex_lock(lock)`
7		`oldVal = bins[b]`
8		`newVal = oldVal + 1`
9		`bins[b] = newVal`
10		`mutex_unlock(lock)`

Thread B cannot acquire the lock before Thread A releases it (or vice versa)

Locks and SIMD Execution

Assume threads 0 and 1 are in the same warp and try to acquire the same lock

```
mutex_lock(lock)

oldVal = bins[b]

...

mutex_unlock(lock)
```

Thread 0	Thread 1
Acquires lock	Waiting for Thread 0 to release lock
Waiting for Thread 1 to complete previous instruction (SIMD)	

Using locks with SIMD execution may cause deadlock

Atomic Operations

- **Atomic operations** perform read-modify-write with a single ISA instruction
- The hardware guarantees that no other thread can access the memory location until the operation completes
- Concurrent atomic operations to the same memory location are serialized by the hardware

Needed When Threads Write to Same Location

- When **two threads may write to** the **same memory** location, the program may **need atomic operations**.
- Sharing is **not always easy to recognize**…
  - Do two insertions into a hash table share data?
  - What about two graph node updates based on all of the nodes' neighbors?
  - What if nodes are on same side of bipartite graph?
- Common failure mode:
  - **Programmer *thinks* operations** are **independent**.
  - Hasn't considered input data for which they are not.
  - Or another programmer reuses code without understanding assumptions that imply independence.
- Also: **atomicity does not constrain relative order**.

Implementing Atomic Operations

- Many ISAs offer synchronization primitives,
  - **instructions with** one (or more) **address operands**
  - **that execute atomically with respect to one another** when used on the same address.
- Mostly read, modify, write operations
  - Bit test and set
  - Compare and swap / exchange
  - Swap / exchange
  - Fetch and increment / add

Atomicity Enforced by Microarchitecture

- When synchronization primitives execute,
  - **hardware ensures** that **no other** thread
  - **accesses** the location **until** the operation is **complete**.

- Other threads that access the location
  - are typically stalled or held in a queue until their turn.
  - **Threads perform atomic operations serially**.

Atomic Compare and Swap (CAS)

- CAS is an atomic instruction used in multithreading to achieve synchronization.
- It compares the contents of a memory location with a given value and, only if they are the same, modifies the contents of that memory location to a new given value. This is done as a single atomic operation.

```
bool atomicCAS(int *address, int old, int new) {
    if (*address != old)
        return false;
    *address = new;
    return true;
}
```

Atomic Add using CAS

- The function performs the action *address ← *address + value atomically and returns the original value stored at address.
- There is no requirement that any sequence of operations is atomic except for atomicCAS.

```
int atomicAdd(int* address, int value){
  bool done = false;
  while (!done) {
    old_v = *address;
    done = atomicCAS(address, old_v, old_v+value);
  }
  return old_v;
}
```

Atomic Operations in CUDA

- Atomic Add:

```
T atomicAdd(T* address, T val)
```

  - `T` can be int, unsigned int, float, double, etc.
  - Reads the value stored at `address`, adds `val` to it, stores the new value at `address`, and returns the old value originally stored
  - Function call translated to a single ISA instruction
    - Such special functions are called *intrinsics*

- Other operations available:
  - sub, min, max, inc, dec, and, or, xor, exchange, compare and swap

Code with Atomic Operations

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
if(i < width*height) {
    unsigned char b = image[i];
    ++bins[b];
}
```

Race condition

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
if(i < width*height) {
    unsigned char b = image[i];
    atomicAdd(&bins[b], 1);
}
```

Histogramming

- A method for extracting notable features and patterns from large data sets
  - Feature extraction for object recognition in images
  - Fraud detection in credit card transactions
  - Correlating heavenly object movements in astrophysics
  - …

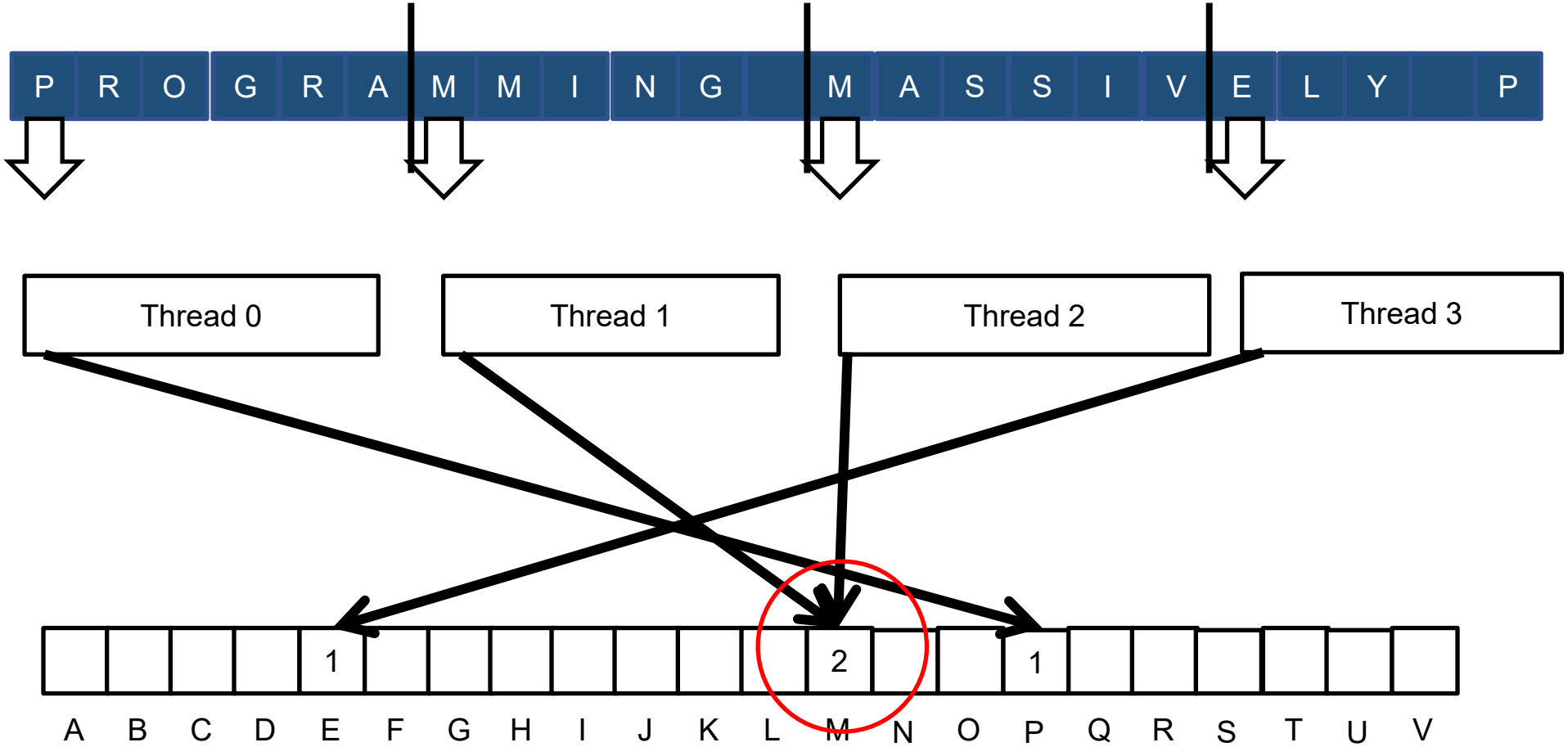
- Basic histograms - for each element in the data set, use the value to identify a “bin” to increment

A Histogram Example

- In sentence “Programming Massively Parallel Processors” build a histogram of frequencies of each letter
- A(4), C(1), E(1), G(1), …

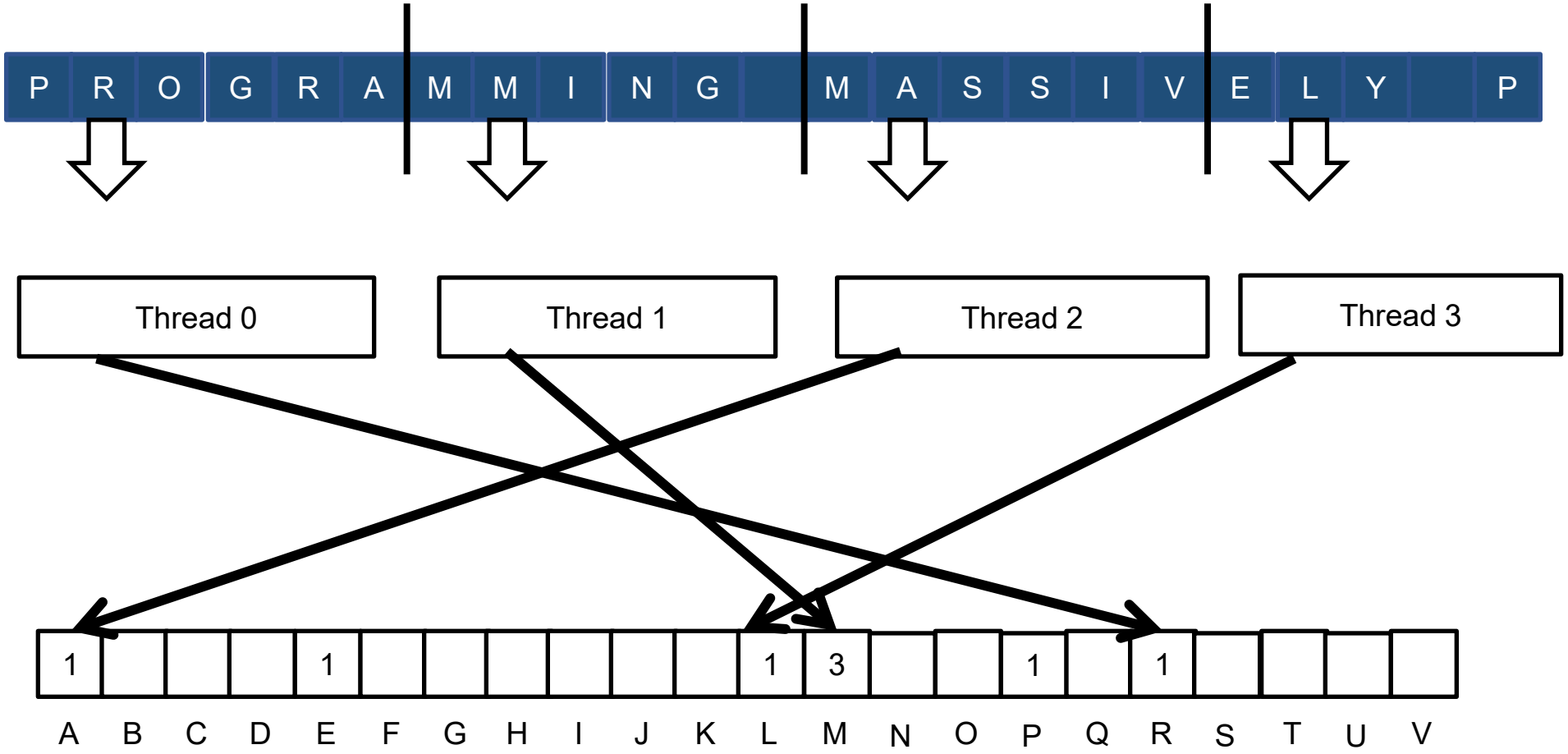
- How do you do this in parallel?

Iteration #1 – 1st letter in each section

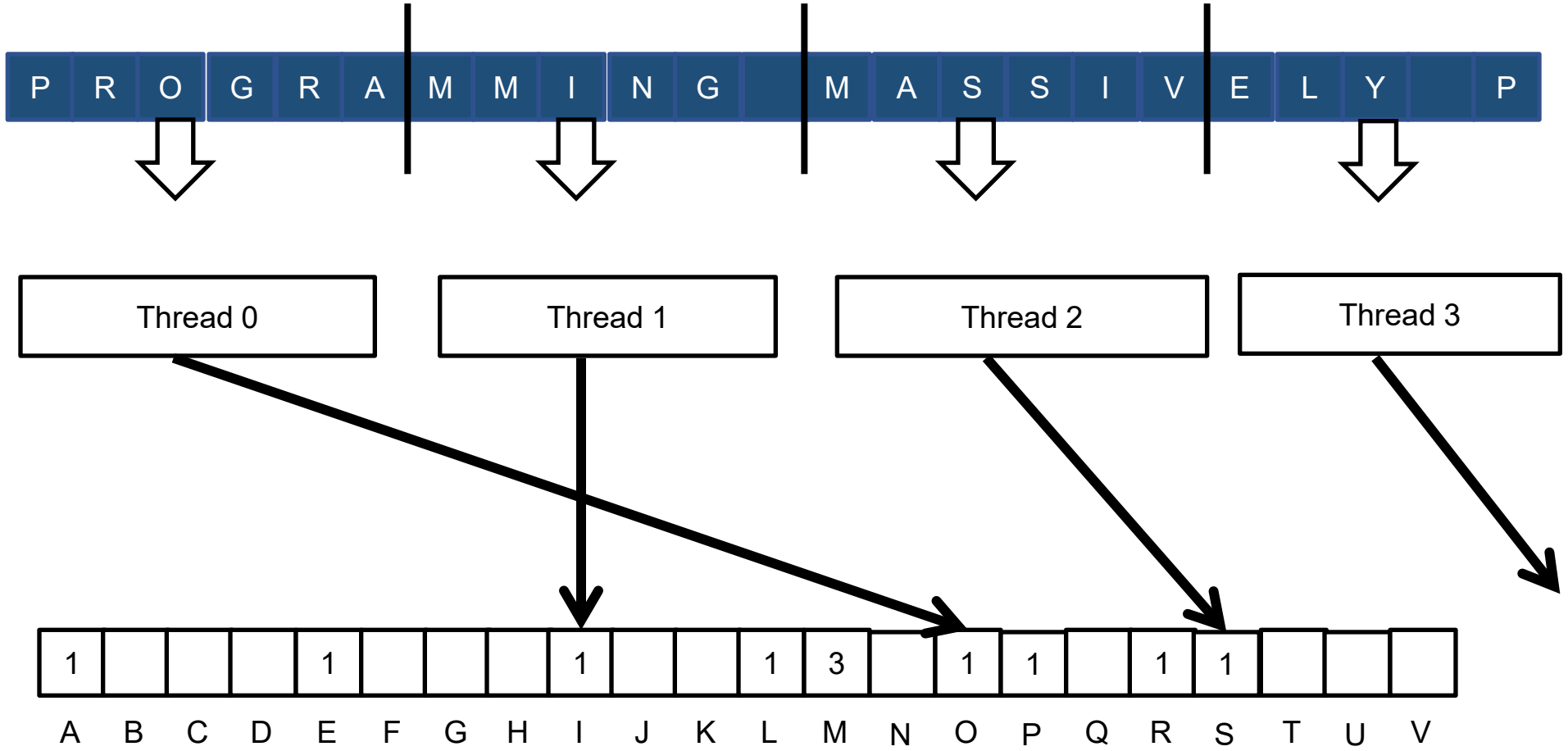


Atomic operation enforces correct update

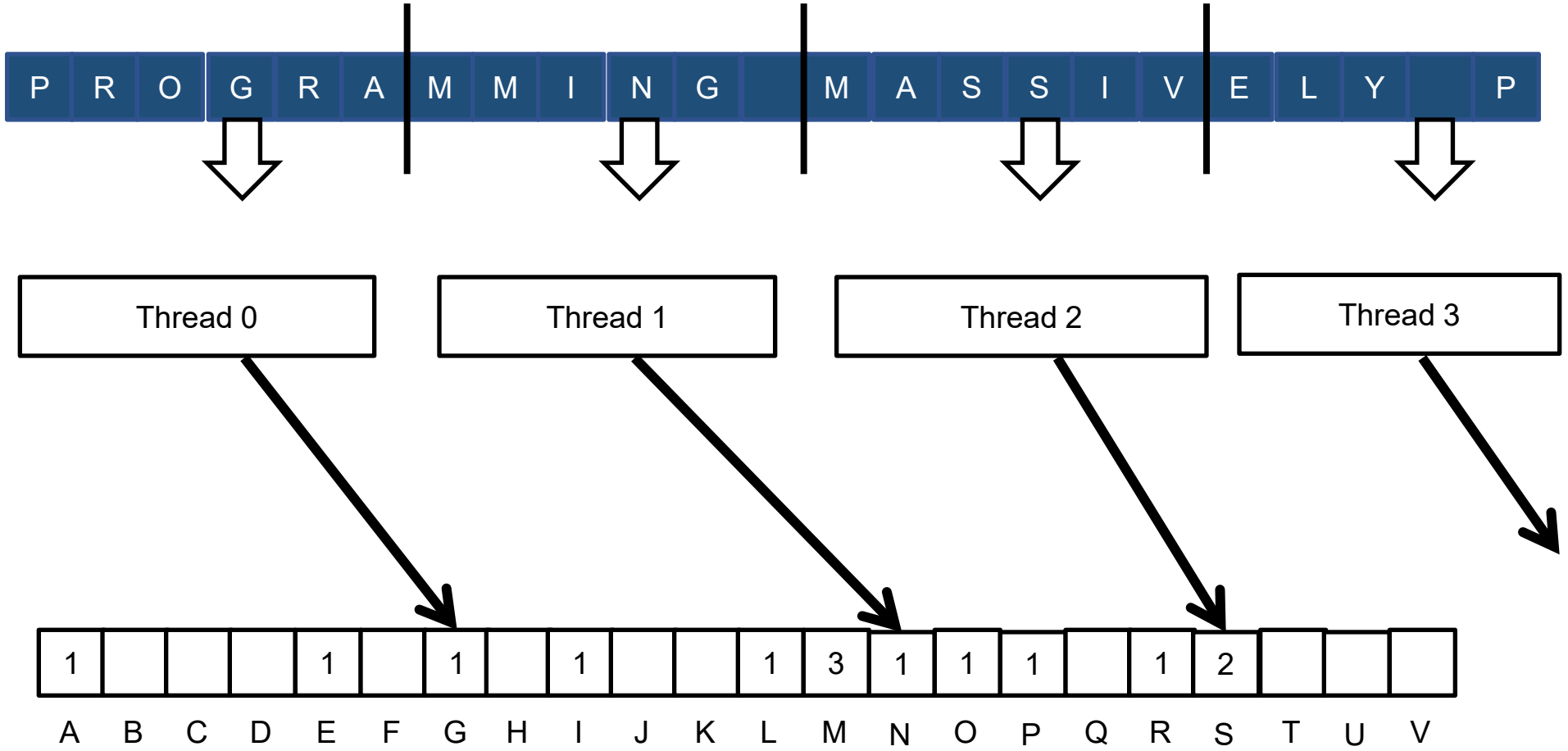
Iteration #2 – 2nd letter in each section



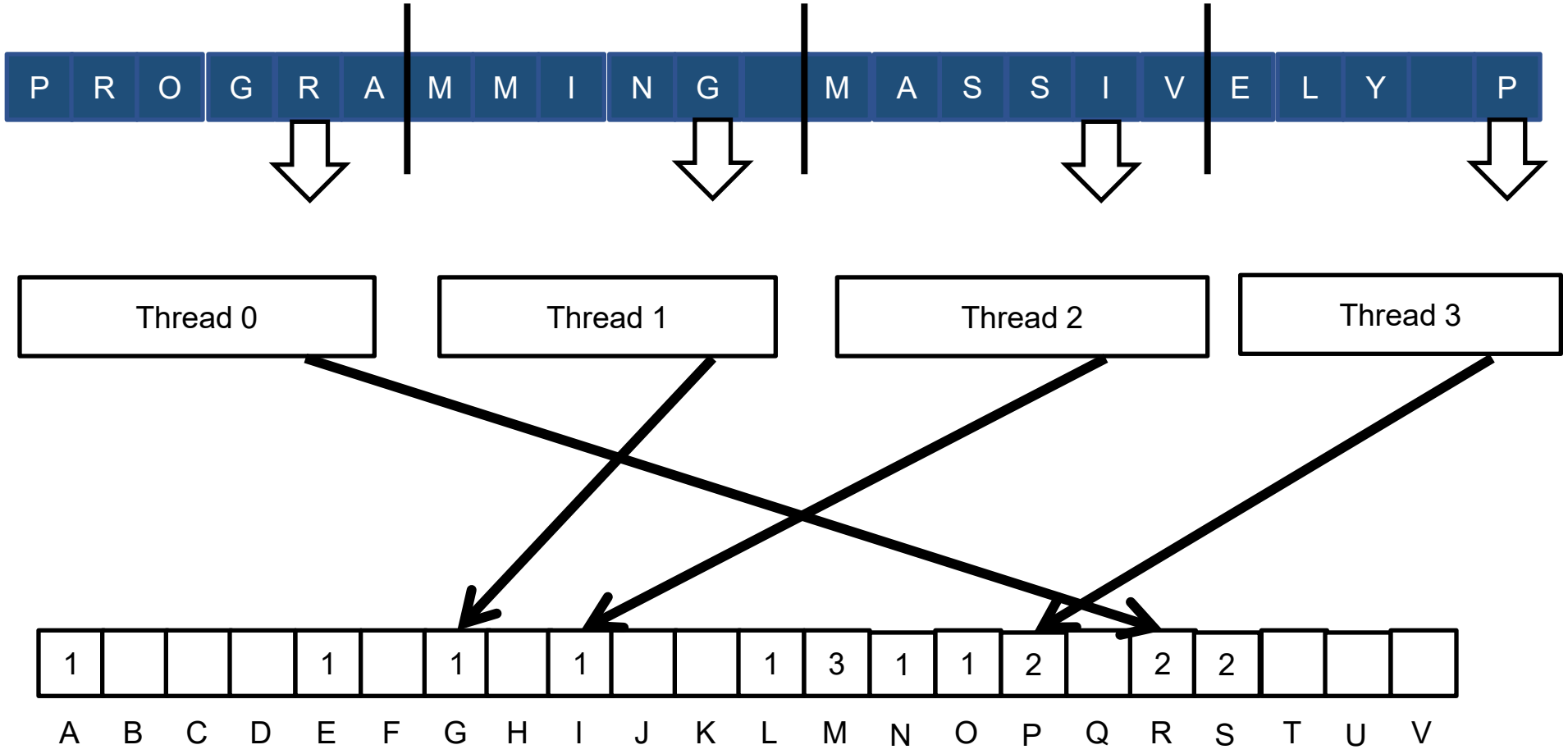
Iteration #3


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 3 1 1 1 1
A B C D E F G H I J K L M N O P Q R S T U V

Iteration #4

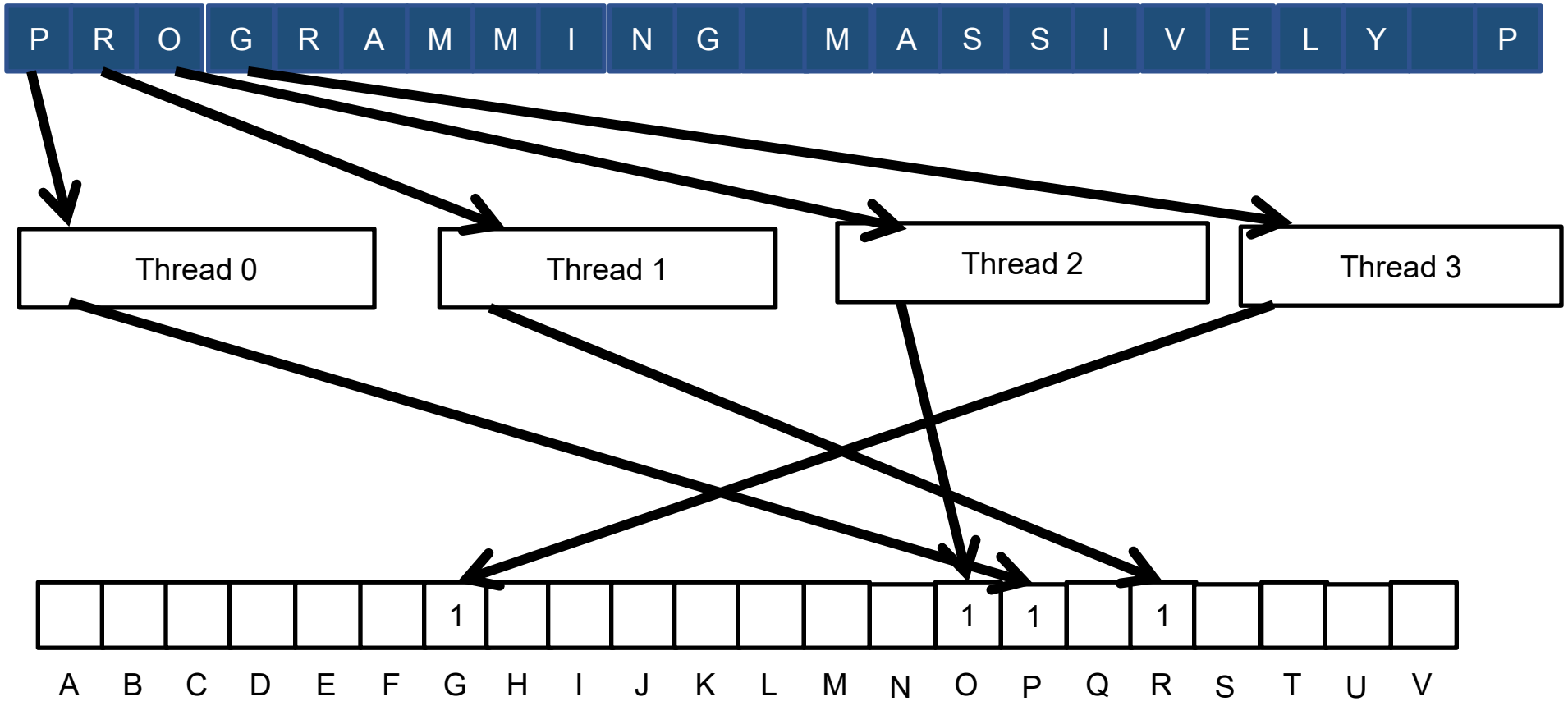

P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 1 3 1 1 1 1 2
A B C D E F G H I J K L M N O P Q R S T U V

Iteration #5


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 1 3 1 1 2 2 2
A B C D E F G H I J K L M N O P Q R S T U V

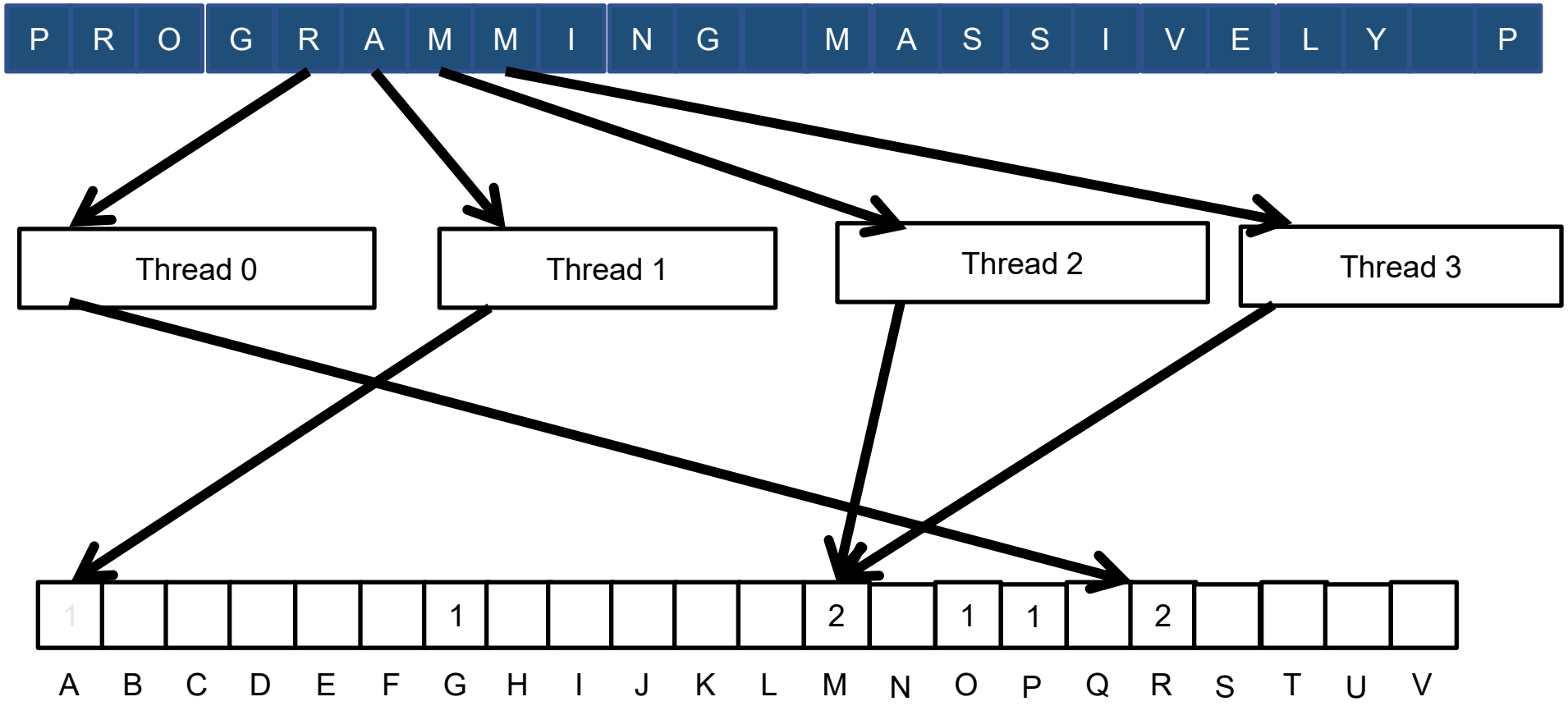
A Better Approach

- Reads from the input array are not coalesced
  - Assign inputs to each thread in a strided pattern
  - Adjacent threads process adjacent input letters



Iteration 2

- All threads move to the next section of input



A Histogram Kernel

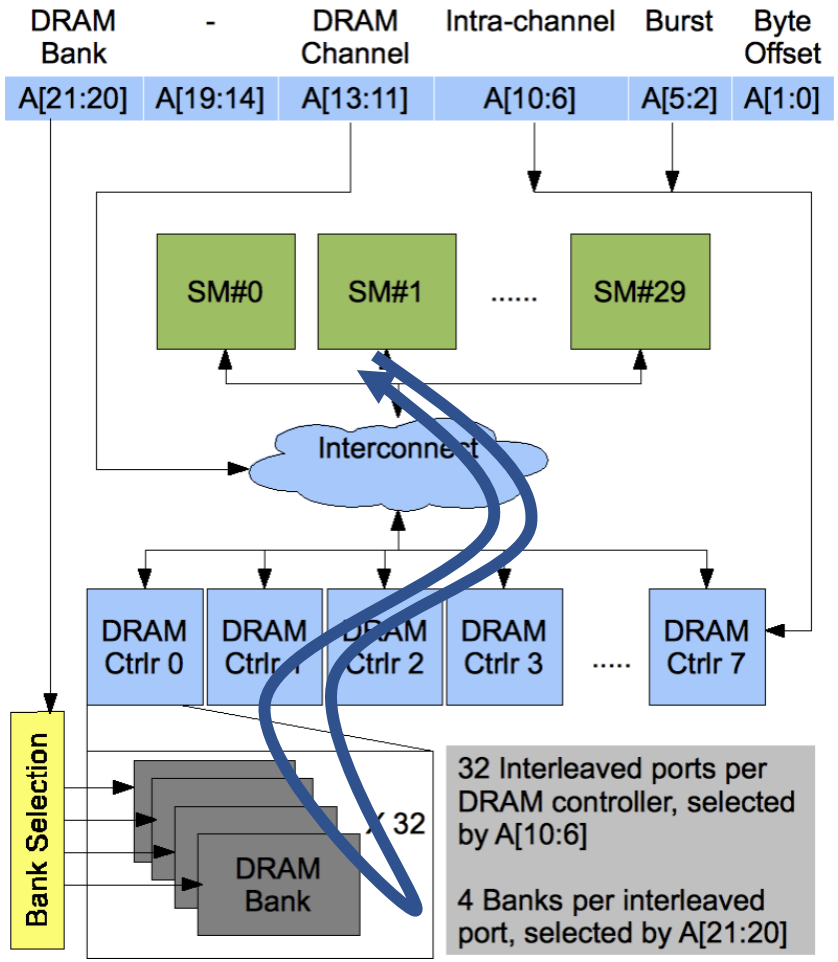
```
__global__ void histo_kernel(char *buf, long size, int *histo)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    // stride = total # of threads
    int stride = blockDim.x * gridDim.x;

    // All threads in the grid collectively handle
    // blockDim.x * gridDim.x consecutive elements

    while (i < size) {
        atomicAdd( &(histo[buf[i]]), 1);
        i += stride;
   }
}
```

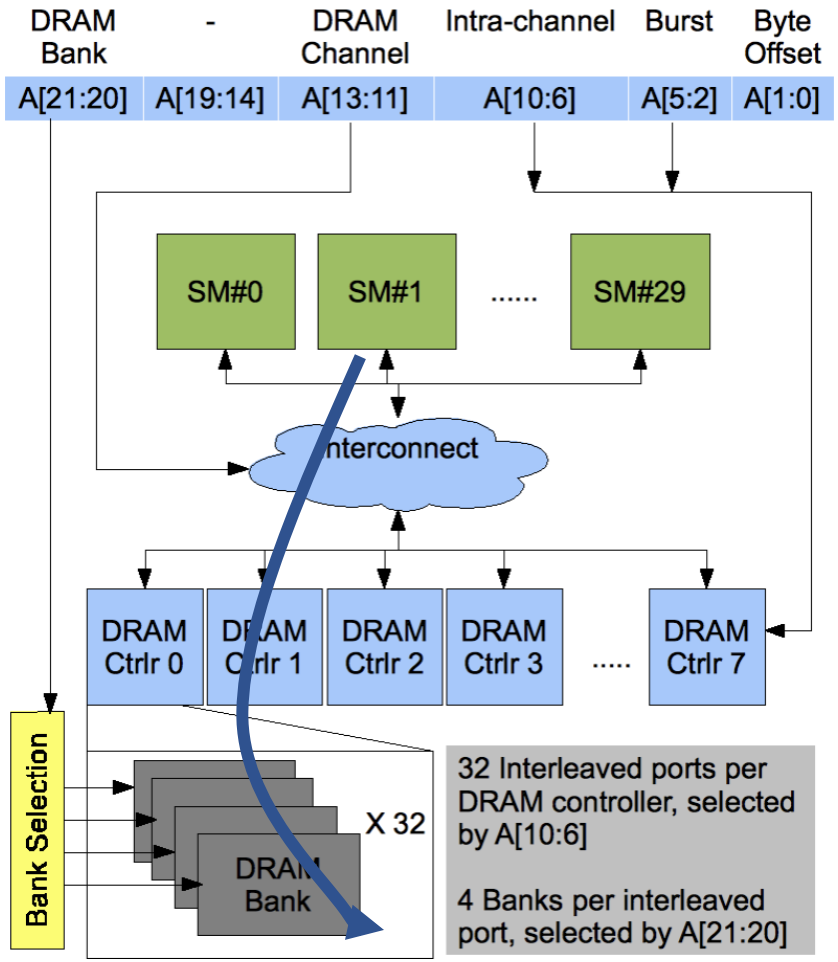
Atomic Operations on DRAM

- An atomic operation starts with a read, with a latency of a few hundred cycles



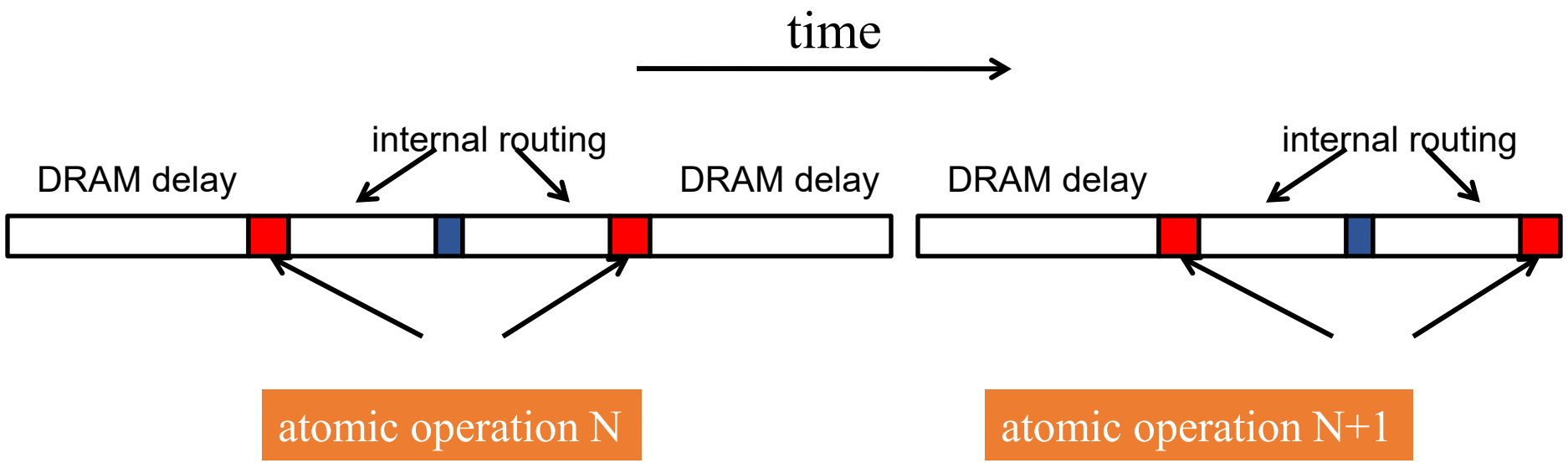
Atomic Operations on DRAM

- An atomic operation starts with a read, with a latency of a few hundred cycles
- The atomic operation ends with a write, with a latency of a few hundred cycles
- During this whole time, no one else can access the location



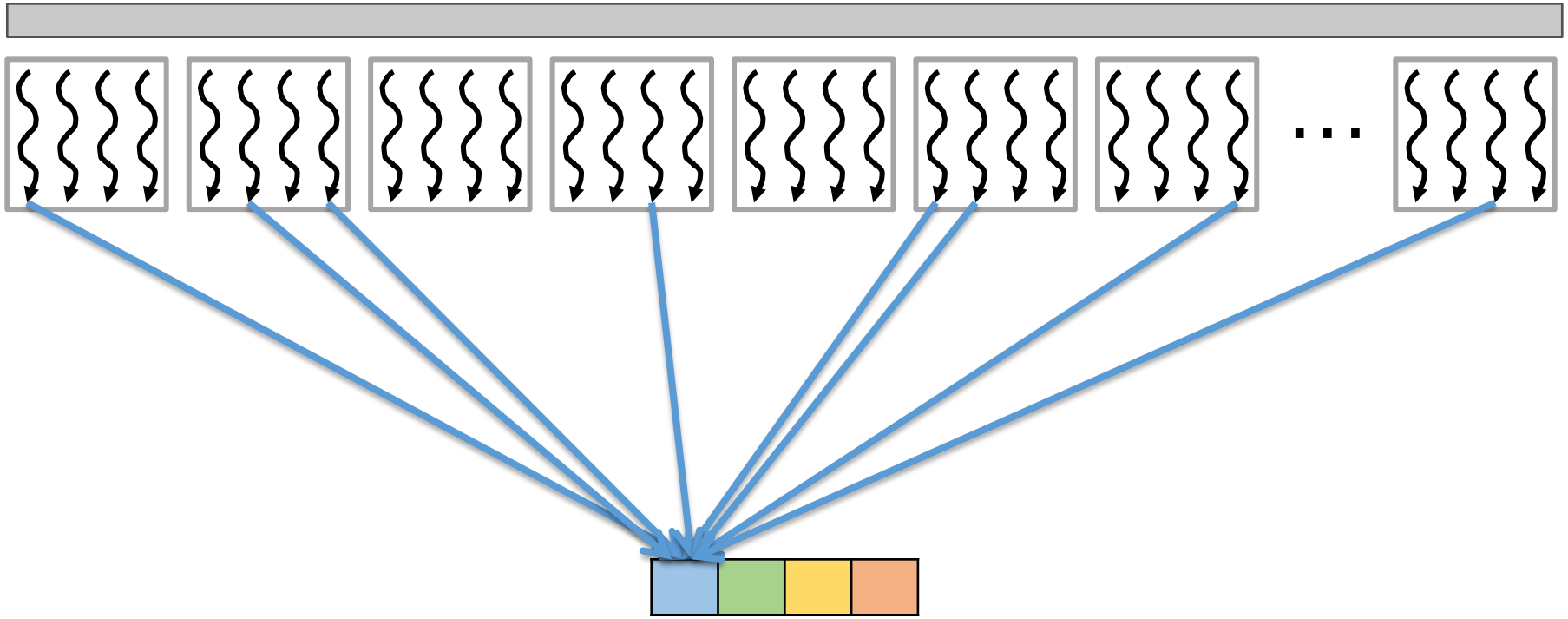
Atomic Operations on DRAM

- Each Load-Modify-Store has two full memory access delays
  - All atomic operations on the same variable (RAM location) are serialized



High Latency

- Atomic operations on global memory have high latency
  - Need to wait for both read and write to complete
  - Need to wait if there are other threads accessing the same location (high probability of contention)



Latency determines throughput of atomic operations

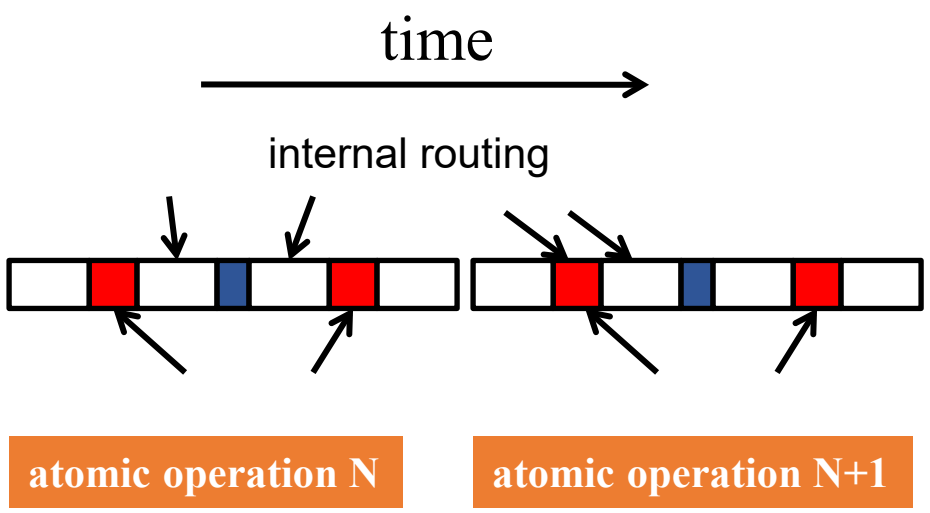
- Throughput of an atomic operation is the rate at which the application can execute an atomic operation on a particular location.
- The rate is limited by the total latency of the read-modify-write sequence, typically more than 1000 cycles for global memory (DRAM) locations.
- This means that if many threads attempt to do atomic operation on the same location (contention), the memory bandwidth is reduced to < 1/1000!

Latency determines throughput of atomic operations

- You may have a similar experience in supermarket checkout
- Some customers realize that they missed an item after they started to check out
- They run to the isle and get the item while the line waits
  - The rate of check is reduced due to the long latency of running to the isle and back.
- Imagine a store where every customer starts the check out before they even fetch any of the items
  - The rate of the checkout will be 1 / (entire shopping time of each customer)

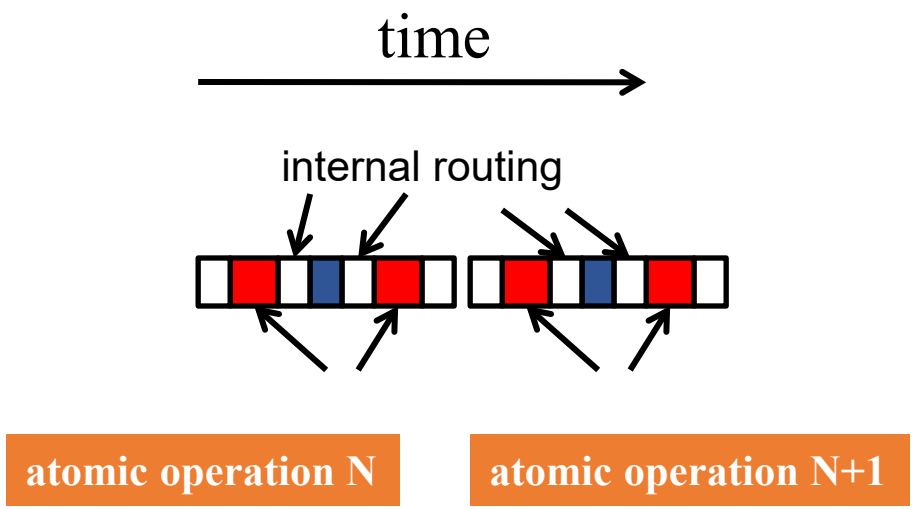
Hardware Improvements

- Atomic operations on L2 cache
  - medium latency, but still serialized
  - Global to all blocks
  - “Free improvement” on Global Memory atomics

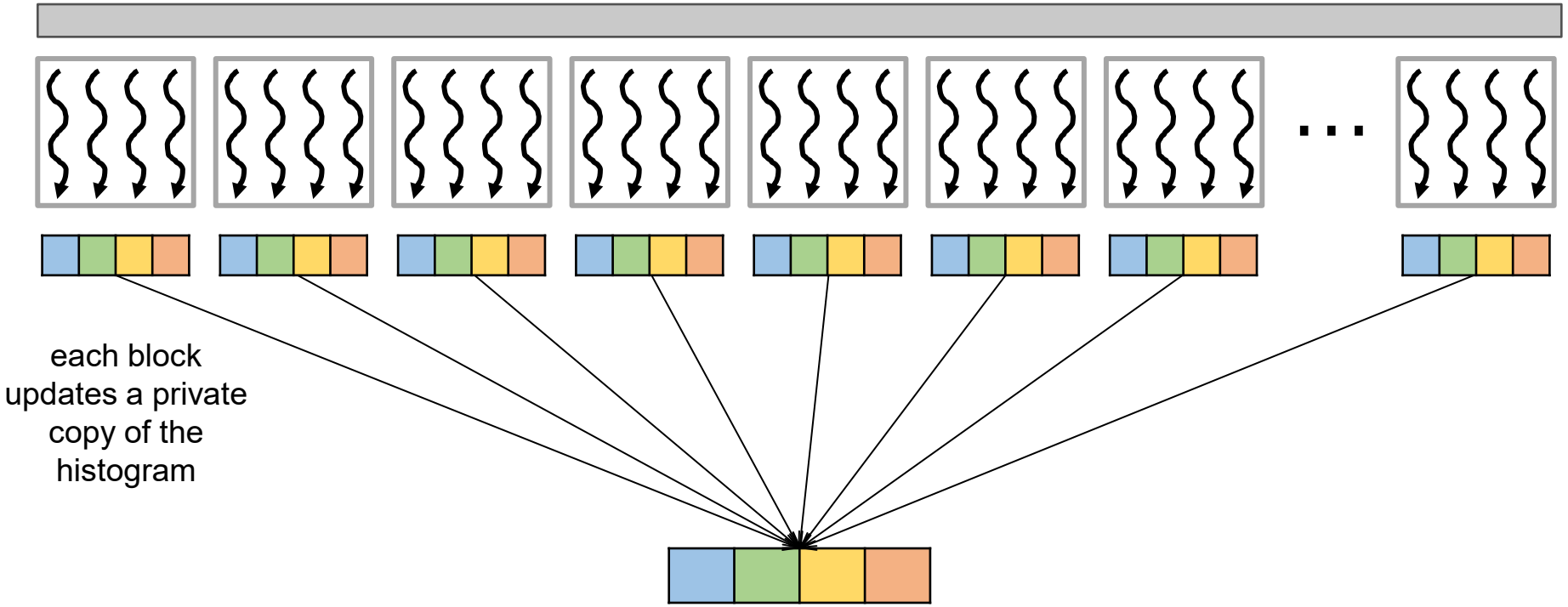


Hardware Improvements

- Atomic operations on Shared Memory
  - Very short latency, but still serialized
  - Private to each thread block
  - Need algorithm work by programmers (more later)



Privatizing the Histogram



Privatization

- **Privatization** is an optimization where multiple private copies of an output are maintained, then the public copy is updated on completion
  - Operations on the output must be associative and commutative because the order of updates has changed

- Advantage: reduces contention on the public copy

Atomics in Shared Memory Requires Privatization

- Create private copies of the histo[] array for each thread block

```
__global__ void histo_kernel(unsigned char *buffer,
                             long size, unsigned int *histo)
{
    __shared__ unsigned int histo_private[256];

      // warning: this will not work correctly if there are fewer than 256 threads!
   if (threadIdx.x < 256)
        histo_private[threadIdx.x] = 0;

   __syncthreads();
```

Build Private Histogram

- Use private copies of the histo[] array to compute

```
int i = threadIdx.x + blockIdx.x * blockDim.x;

// stride is total number of threads
int stride = blockDim.x * gridDim.x;

while (i < size) {
     atomicAdd( &(private_histo[buffer[i]), 1);
     i += stride;
}
```

Build Final Histogram

- Copy from the histo[] arrays from each thread block to global memory

```
    // wait for all other threads in the block to finish
    __syncthreads();

    if (threadIdx.x < 256)
        atomicAdd( &(histo[threadIdx.x]),
          private_histo[threadIdx.x] );
}
```

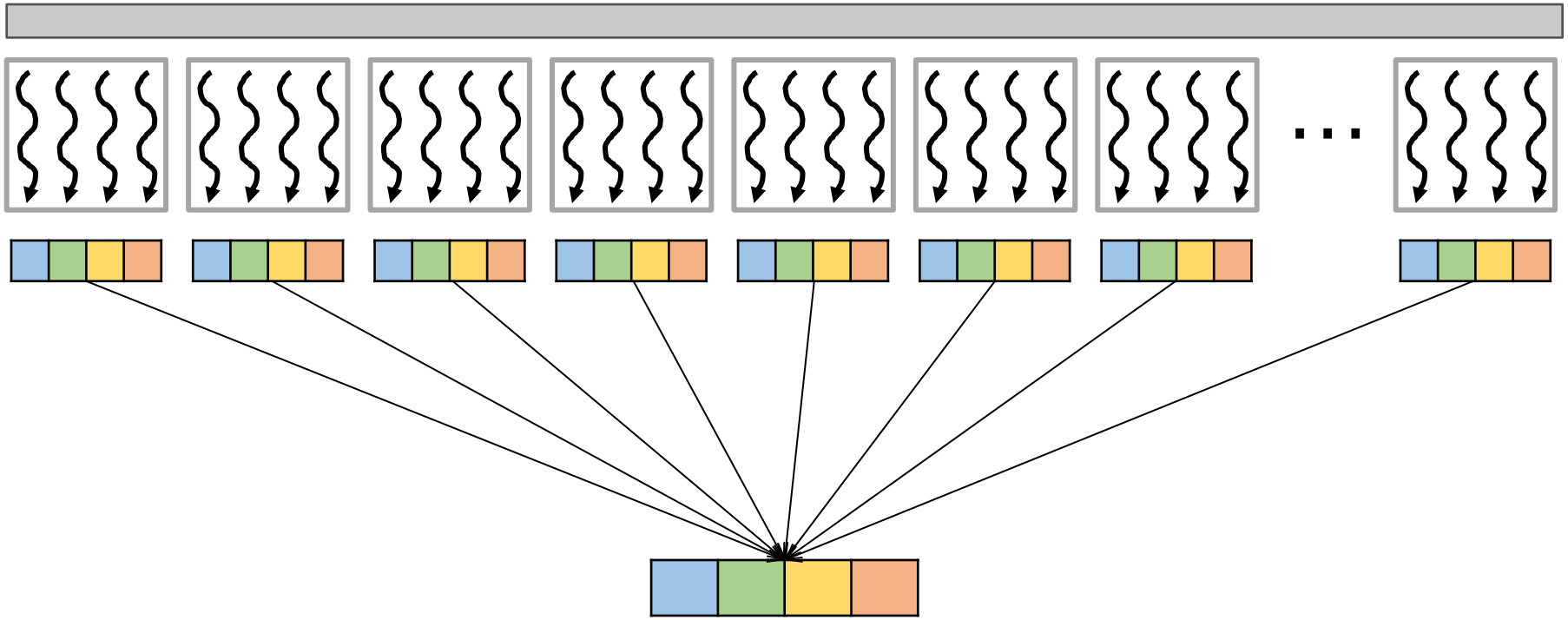
More on Privatization

- Privatization is a powerful and frequently used techniques for parallelizing applications
- The operation needs to be associative and commutative
  - Histogram add operation is associative and commutative
- The histogram size needs to be small
  - Fits into shared memory
- What if the histogram is too large to privatize?

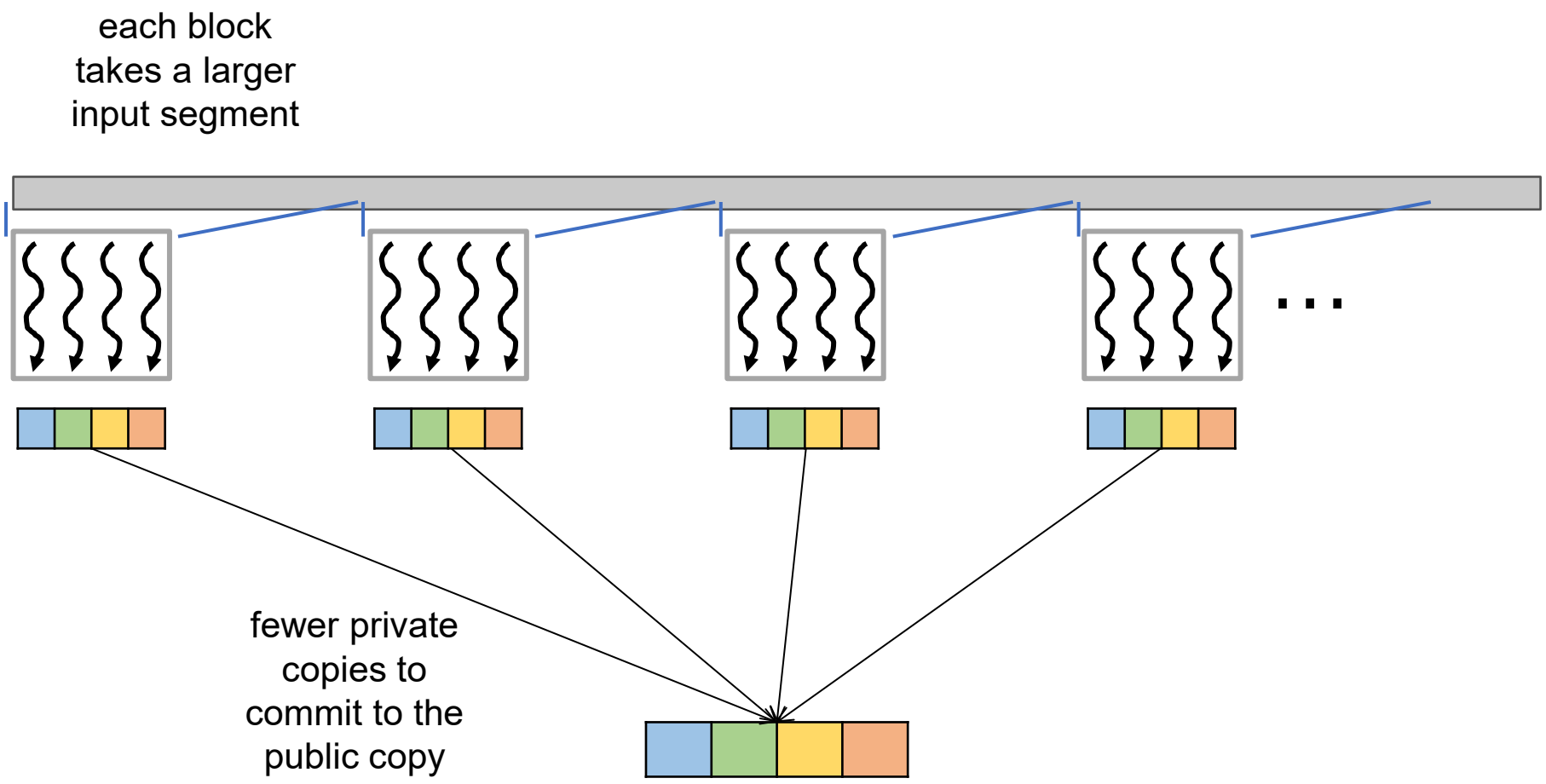
Other Optimization Opportunities

- Are memory accesses coalesced?
  - Yes (for input data)
- Is there control divergence?
  - Negligible (only at the boundary check)
- Is there data reuse?
  - Input data: No
  - Output data: Yes
    - Multiple threads in a block may update the same bin in the private histogram
    - Place the private histogram in shared memory (if it fits)
- Is there an overhead for parallelizing the work?
  - Atomic updates from the private copies to the public copy happen once per block
  - Use thread coarsening to reduce the number of blocks/updates

Before Coarsening

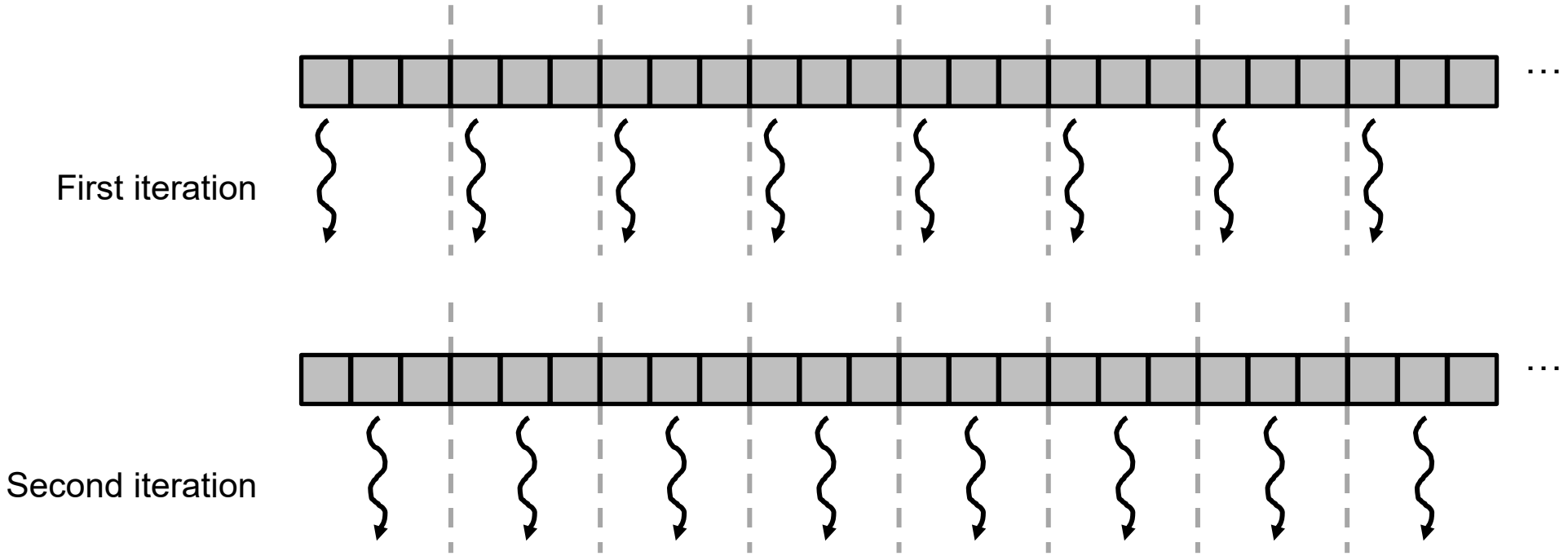


After Coarsening



Coarsening Strategy

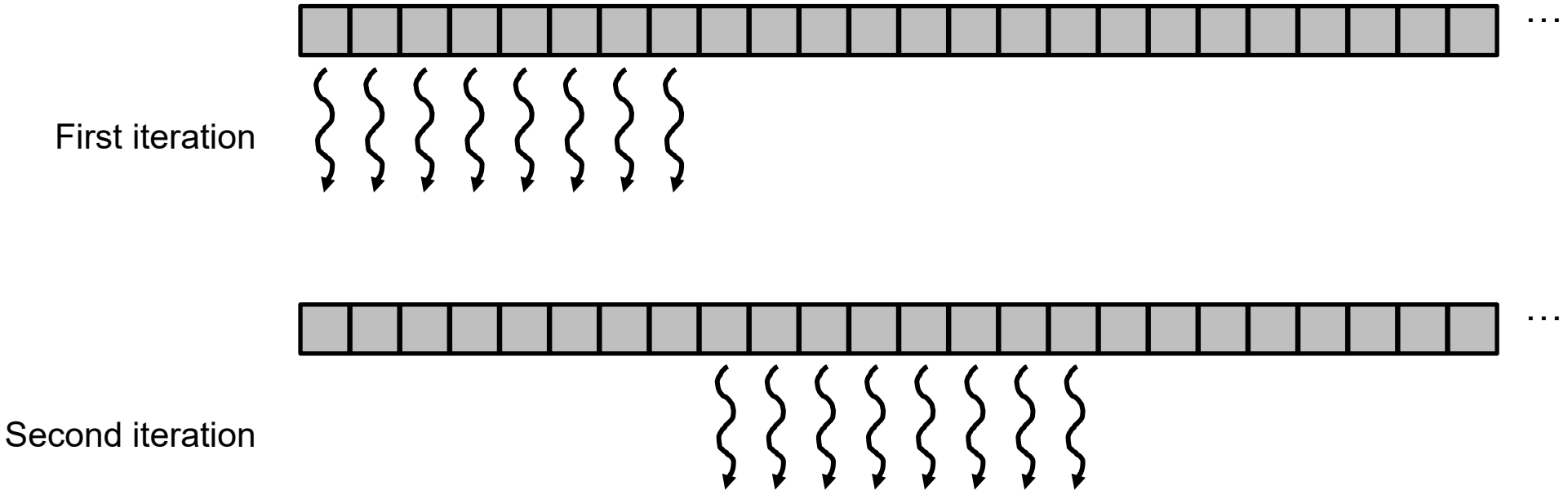
- How to assign multiple inputs to each thread?



Accesses are not coalesced

Coarsening Strategy

- How to assign multiple inputs to each thread?



Accesses are coalesced

Any More Questions?

Things to Read:

Textbook chapter 9

CUDA PG: 7.14. Atomic Functions

Things to Do:

Do not forget to watch recorded lecture 13

Lab 5

Problem Solving

- Q: Suppose a processor supports atomic operations in L2 cache. Assume that each atomic operation takes 5ns to complete in L2 cache and 120ns to complete in DRAM. The kernel performs 20 floating-point operations per atomic operation, and a floating-point operation takes 1ns. Assume the time of L2 atomic operations, DRAM atomic operations, and floating-point operations in each thread do not overlap. The floating-point throughput of the kernel execution is 0.2424 GFLOPS, and every thread in a block performs 5 atomic operations and 100 floating-point operations. What percent of the atomic operations happened in L2 cache?
- A:
  - 50%

Explanation

- The total time taken by a thread for all operations, assuming all atomic operations are in L2 cache, is: 5 atomic operations * 5ns/atomic operation + 100 floating-point operations * 1ns/floating-point operation = 25ns + 100ns = 125ns
- Similarly, the total time taken by a thread for all operations, assuming all atomic operations are in DRAM, is: 5 atomic operations * 120ns/atomic operation + 100 floating-point operations * 1ns/floating-point operation = 600ns + 100ns = 700ns
- Let's denote the percentage of atomic operations that happened in DRAM as x. Therefore, the percentage of atomic operations that happened in L2 cache is $1 - x$ and the total execution time of a thread is: $(1 - x)$ * 125ns/thread + x * 700ns/thread
- Given that the floating-point throughput of the kernel execution is 0.2424 GFLOPS, we can calculate the total time taken for all operations in a thread (1 GFLOP = 1 billion floating-point operations): 0.2424 GFLOPS = 0.2424 * 1 billion floating-point operations/second = 242.4 million floating-point operations/second.
- Since every thread performs 100 floating-point operations, the number of threads that can be executed per second is: 242.4 million threads/second = 242.4 million floating-point operations/second / 100 floating-point operations/thread = 2.424 million threads/second.
- Therefore, the total time taken for all operations in a thread is: 1 / 2.424 million threads/second = 0.0000004125 seconds/thread = 412.5ns/thread.
- Thus: $(1 - x)$ * 125ns/thread + x * 700ns/thread = 412.5ns/thread. Solving this for x gives us 50%.